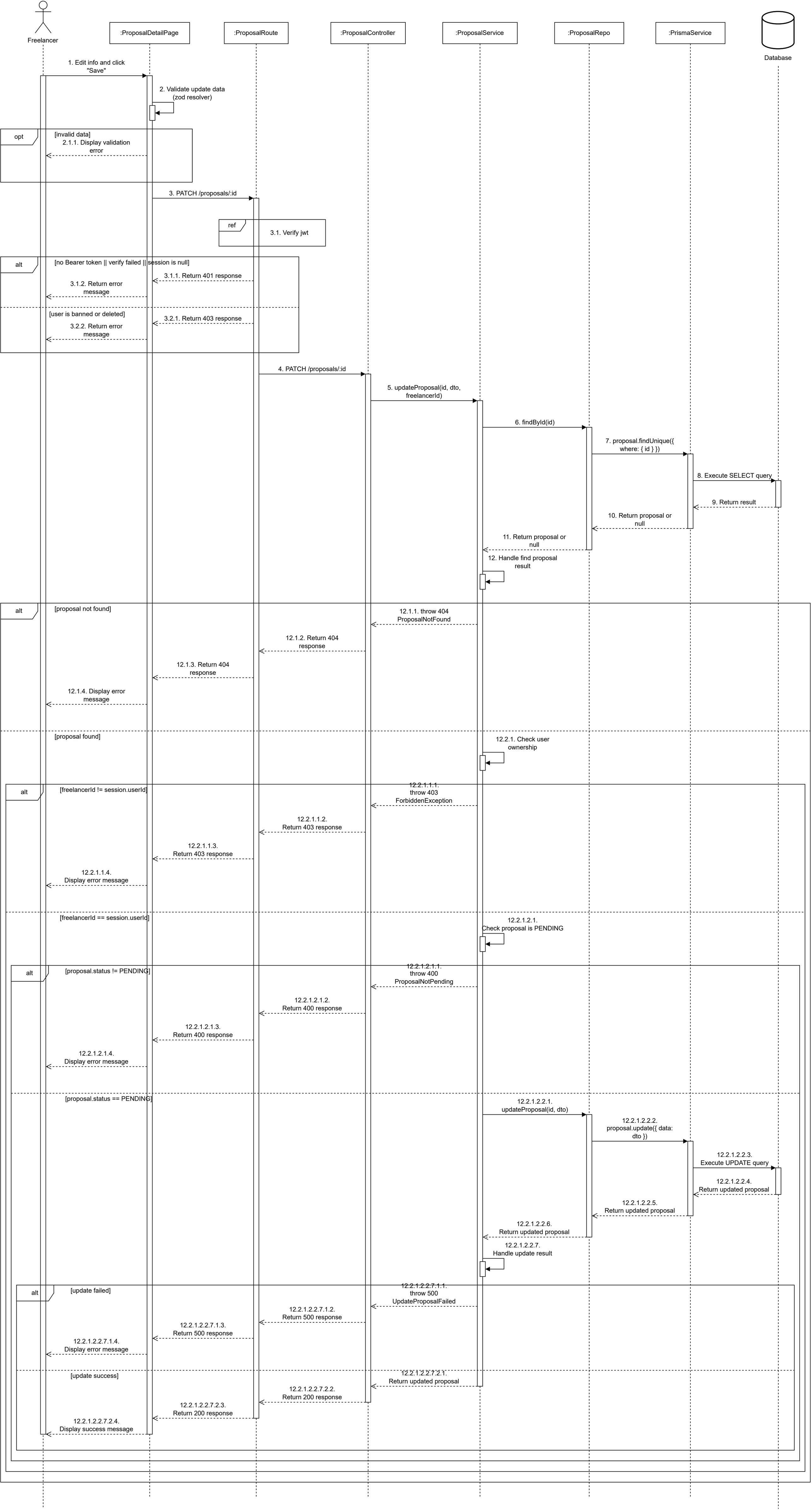

Freelancer
:ProposalDetailPage
:ProposalRoute
:ProposalController
:ProposalService
:ProposalRepo
:PrismaService
Database
1. Edit info and click "Save"
2. Validate update data (zod resolver)
opt
[invalid data]
2.1.1. Display validation error
3. PATCH /proposals/:id
ref
3.1. Verify jwt
alt
[no Bearer token || verify failed || session is null]
3.1.1. Return 401 response
3.1.2. Return error message
[user is banned or deleted]
3.2.1. Return 403 response
3.2.2. Return error message
4. PATCH /proposals/:id
5. updateProposal(id, dto, freelancerId)
6. findById(id)
7. proposal.findUnique({ where: { id } })
8. Execute SELECT query
9. Return result
10. Return proposal or null
11. Return proposal or null
12. Handle find proposal result
alt
[proposal not found]
12.1.1. throw 404 ProposalNotFound
12.1.2. Return 404 response
12.1.3. Return 404 response
12.1.4. Display error message
[proposal found]
12.2.1. Check user ownership
alt
[freelancerId != session.userId]
12.2.1.1.1. throw 403 ForbiddenException
12.2.1.1.2. Return 403 response
12.2.1.1.3. Return 403 response
12.2.1.1.4. Display error message
[freelancerId == session.userId]
12.2.1.2.1. Check proposal is PENDING
alt
[proposal.status != PENDING]
12.2.1.2.1.1. throw 400 ProposalNotPending
12.2.1.2.1.2. Return 400 response
12.2.1.2.1.3. Return 400 response
12.2.1.2.1.4. Display error message
[proposal.status == PENDING]
12.2.1.2.2.1. updateProposal(id, dto)
12.2.1.2.2.2. proposal.update({ data: dto })
12.2.1.2.2.3. Execute UPDATE query
12.2.1.2.2.4. Return updated proposal
12.2.1.2.2.5. Return updated proposal
12.2.1.2.2.6. Return updated proposal
12.2.1.2.2.7. Handle update result
alt
[update failed]
12.2.1.2.2.7.1.1. throw 500 UpdateProposalFailed
12.2.1.2.2.7.1.2. Return 500 response
12.2.1.2.2.7.1.3. Return 500 response
12.2.1.2.2.7.1.4. Display error message
[update success]
12.2.1.2.2.7.2.1. Return updated proposal
12.2.1.2.2.7.2.2. Return 200 response
12.2.1.2.2.7.2.3. Return 200 response
12.2.1.2.2.7.2.4. Display success message